IOT · LAB3

Làm việc với I²C — Bài tập

Phiếu bài tập & Mẫu nộp bài của sinh viên — DHT20 & LCD với FreeRTOS trên YoloUNO (ESP32-S3)

Môn học: Hệ thống nhúng — Từ Arduino Uno đến ESP32-S3 và Raspberry Pi kết hợp AI · **Nền tảng:** YoloUNO (ESP32-S3)

Thông tin sinh viên

Họ và tên	Phạm Công Võ
MSSV	2313946
Lớp / Nhóm	L01
Ngày	19/11/2026

Cách hoàn thành & nộp tài liệu này

Tài liệu này là mẫu nộp bài cho các bài tập LAB 3. Hãy đọc các bước dưới đây trước khi bắt đầu.

1. Điền thông tin sinh viên vào bảng phía trên (họ tên, MSSV, lớp và ngày).
2. Giải từng bài tập trên phần cứng thật trước. Build và nạp mã lên YoloUNO (với DHT20 và LCD (tùy chọn) cắm vào các cổng I²C màu ĐỎ) và xác nhận mạch hoạt động đúng yêu cầu.
3. Với mỗi bài tập, dán **mã nguồn hoàn chỉnh, chạy được** của bạn (các file liên quan — ví dụ `main.cpp` và mọi file tác vụ mới) vào khung được cung cấp ngay bên dưới bài tập đó.
4. Tạo mọi tác vụ mới bên trong `setup()` bằng `xTaskCreate()`, và dùng các biến toàn cục dùng chung `glob_temperature` và `glob_humidity` để giao tiếp với tác vụ cảm biến. Giữ nguyên tên tác vụ được cho trong từng bài tập (ví dụ `task_neopixel`).
5. Khi bài tập yêu cầu bạn **giải thích** một lựa chọn (Bài tập 4 và 5), hãy viết phần giải thích vào khung giải thích được cung cấp.
6. Không xóa hoặc thay đổi kích thước các khung trả lời — khung sẽ tự mở rộng khi bạn dán thêm dòng.
7. Khi đã hoàn thành cả năm bài tập, lưu file này (giữ định dạng `.docx`, hoặc xuất ra `PDF`) và nộp **toàn bộ tài liệu**.

> **Lưu ý — cách các tác vụ chia sẻ dữ liệu**
> Bài thực hành này được xây dựng dựa trên project DHT20 + LCD. Tác vụ temp_humi_monitor cập nhật glob_temperature và glob_humidity mỗi 5 giây; các tác vụ mới của bạn (NeoPixel, LED, bộ lọc) đọc các biến toàn cục đó để quyết định hành động. Mọi tác vụ chạy đồng thời, vì vậy nhiều bài tập được thiết kế để hoạt động song song.

BÀI TẬP 1 NeoPixel làm đèn chỉ thị giá trị

Yêu cầu

- Sửa mã để thêm một LED NeoPixel.
- Viết một tác vụ (`task_neopixel`) thay đổi màu NeoPixel theo nhiệt độ:
  - **Xanh dương** nếu **TEMP < 20°C**
  - **Xanh lá** nếu **20°C ≤ TEMP < 28°C**
  - **Đỏ** nếu **TEMP ≥ 28°C**
- Màu phải được cập nhật mỗi khi có giá trị đọc mới từ cảm biến (**mỗi 5 giây**).

Lời giải của bạn — mã nguồn hoàn chỉnh:

```
#include <Adafruit_NeoPixel.h>
#include <Arduino.h>
#include <DHT20.h>
#include <LiquidCrystal_I2C.h>
#include <Wire.h>

#define LED_PIN GPIO_NUM_48
#define NEO_PIN 45
#define LED_COUNT 1
#define SDA_PIN 11
#define SCL_PIN 12
#define LCD_ADDR 33

DHT20 dht20;
LiquidCrystal_I2C lcd(LCD_ADDR, 16, 2);
Adafruit_NeoPixel strip(LED_COUNT, NEO_PIN, NEO_GRB + NEO_KHZ800);

volatile float glob_temperature = 0.0;
volatile float glob_humidity = 0.0;

// Tác vụ đọc cảm biến DHT20 và hiển thị LCD
void task_temp_humi_monitor(void *pvParameters) {
  Wire.begin(SDA_PIN, SCL_PIN);
  dht20.begin();

  lcd.begin();
  lcd.backlight();
  lcd.clear();

  while (1) {
    int status = dht20.read();
    float t = dht20.getTemperature();
    float h = dht20.getHumidity();

    if (status == 0 && !isnan(t) && !isnan(h)) {
      glob_temperature = t;
      glob_humidity = h;

      Serial.print("[DHT20] Temp: ");
      Serial.print(glob_temperature, 1);
      Serial.print(" °C | Humi: ");
      Serial.print(glob_humidity, 1);
```

```
      Serial.println(" %");

      lcd.setCursor(0, 0);
      lcd.print("Temp: ");
      lcd.print(glob_temperature, 1);
      lcd.print((char)223);
      lcd.print("C   ");

      lcd.setCursor(0, 1);
      lcd.print("Humi: ");
      lcd.print(glob_humidity, 1);
      lcd.print("%   ");
    }
    vTaskDelay(5000);
  }
}

// Tác vụ đổi màu NeoPixel theo dải nhiệt độ
void task_neopixel(void *pvParameters) {
  strip.begin();
  strip.clear();
  strip.show();

  while (1) {
    float temp = glob_temperature;

    if (temp < 20.0) {
      strip.setPixelColor(0, strip.Color(0, 0, 255)); // Xanh dương
    } else if (temp < 28.0) {
      strip.setPixelColor(0, strip.Color(0, 255, 0)); // Xanh lá
    } else {
      strip.setPixelColor(0, strip.Color(255, 0, 0)); // Đỏ nếu
    }
    strip.show();
    vTaskDelay(1000);
  }
}

void setup() {
  Serial.begin(115200);

  xTaskCreate(task_temp_humi_monitor,
              "Temp_Humi_Monitor",
              4096,
              NULL,
              2,
              NULL
  );
  xTaskCreate(task_neopixel,
              "Task_NeoPixel",
              2048,
              NULL,
              1,
              NULL
  );
}
void loop() {}
```

BÀI TẬP 2 LED nhấp nháy theo độ ẩm

Yêu cầu

- Mở rộng hệ thống để tốc độ nhấp nháy của LED phản ánh giá trị độ ẩm:
  - **Độ ẩm thấp (<40%)**: LED nhấp nháy mỗi **2 s**.
  - **Độ ẩm bình thường (40%–60%)**: LED nhấp nháy mỗi **1 s**.
  - **Độ ẩm cao (>60%)**: LED nhấp nháy nhanh mỗi **0,5 s**.
- Hiện thực chức năng này thành một **tác vụ riêng** giao tiếp với tác vụ cảm biến (dùng biến toàn cục).
- Lưu ý: hành vi đổi màu NeoPixel của Bài tập 1 phải tiếp tục hoạt động **song song** với bài tập này.

Lời giải của bạn — mã nguồn hoàn chỉnh:

```
#include <Adafruit_NeoPixel.h>
#include <Arduino.h>
#include <DHT20.h>
#include <LiquidCrystal_I2C.h>
#include <Wire.h>

#define LED_PIN GPIO_NUM_48
#define NEO_PIN 45
#define LED_COUNT 1
#define SDA_PIN 11
#define SCL_PIN 12
#define LCD_ADDR 33

DHT20 dht20;
LiquidCrystal_I2C lcd(LCD_ADDR, 16, 2);
Adafruit_NeoPixel strip(LED_COUNT, NEO_PIN, NEO_GRB + NEO_KHZ800);

volatile float glob_temperature = 0.0;
volatile float glob_humidity = 0.0;

void task_temp_humi_monitor(void *pvParameters) {
  Wire.begin(SDA_PIN, SCL_PIN);
  dht20.begin();

  lcd.begin();
  lcd.backlight();
  lcd.clear();

  while (1) {
    int status = dht20.read();
    float t = dht20.getTemperature();
    float h = dht20.getHumidity();

    if (status == 0 && !isnan(t) && !isnan(h)) {
      glob_temperature = t;
      glob_humidity = h;

      lcd.setCursor(0, 0);
      lcd.print("Temp: ");
      lcd.print(glob_temperature, 1);
      lcd.print((char)223);
      lcd.print("C   ");
```

```
      lcd.setCursor(0, 1);
      lcd.print("Humi: ");
      lcd.print(glob_humidity, 1);
      lcd.print("%   ");
    }
    vTaskDelay(5000);
  }
}

void task_neopixel(void *pvParameters) {
  strip.begin();
  strip.clear();
  strip.show();

  while (1) {
    float temp = glob_temperature;

    if (temp < 20.0) {
      strip.setPixelColor(0, strip.Color(0, 0, 255));
    } else if (temp < 28.0) {
      strip.setPixelColor(0, strip.Color(0, 255, 0));
    } else {
      strip.setPixelColor(0, strip.Color(255, 0, 0));
    }
    strip.show();
    vTaskDelay(1000);
  }
}

// Tác vụ điều khiển LED nhấp nháy theo độ ẩm
void task_led(void *pvParameters) {
  pinMode(LED_PIN, OUTPUT);

  while (1) {
    float humi = glob_humidity;
    uint32_t half_period_ms = 500;

    if (humi < 40.0) {        // Độ ẩm thấp (<40%): nhấp nháy mỗi 2s
      half_period_ms = 1000;
    } else if (humi <= 60.0) {// Bình thường (40%-60%): nhấp nháy mỗi 1s
      half_period_ms = 500;
    } else {                  // Độ ẩm cao (>60%): nhấp nháy nhanh mỗi 0.5s
      half_period_ms = 250;
    }

    digitalWrite(LED_PIN, HIGH);
    vTaskDelay(half_period_ms);

    digitalWrite(LED_PIN, LOW);
    vTaskDelay(half_period_ms);
  }
}

void setup() {
  Serial.begin(115200);
```

```
  xTaskCreate(task_temp_humi_monitor,
              "Temp_Humi_Monitor",
              4096,
              NULL,
              2,
              NULL
  );

  xTaskCreate(task_neopixel,
              "Task_NeoPixel",
              2048,
              NULL,
              1,
              NULL
  );

  xTaskCreate(task_led,
              "Task_LED_Humi",
              2048,
              NULL,
              1,
              NULL
  );
}

void loop() {}
```

BÀI TẬP 3 Xử lý lỗi và phản hồi bằng hình ảnh

Yêu cầu

- Mô phỏng lỗi cảm biến: khi **cả nhiệt độ và độ ẩm đều bằng 0**. Khi điều này xảy ra:
  - Đặt NeoPixel ở trạng thái **vàng, nhấp nháy**, để báo lỗi.
  - LED phải **sáng liên tục** cho đến khi hết lỗi.
- Khi cảm biến hoạt động bình thường trở lại, màu sắc và kiểu nhấp nháy phải **trở về bình thường**.

Lời giải của bạn — mã nguồn hoàn chỉnh:

```
#include <Adafruit_NeoPixel.h>
#include <Arduino.h>
#include <DHT20.h>
#include <LiquidCrystal_I2C.h>
#include <Wire.h>

#define LED_PIN GPIO_NUM_48
#define NEO_PIN 45
#define LED_COUNT 1
#define SDA_PIN 11
#define SCL_PIN 12
#define LCD_ADDR 33

DHT20 dht20;
LiquidCrystal_I2C lcd(LCD_ADDR, 16, 2);
Adafruit_NeoPixel strip(LED_COUNT, NEO_PIN, NEO_GRB + NEO_KHZ800);

volatile float glob_temperature = 0.0;
volatile float glob_humidity = 0.0;
volatile bool glob_sensor_error = false; // Cờ báo lỗi cảm biến

void task_temp_humi_monitor(void *pvParameters) {
  Wire.begin(SDA_PIN, SCL_PIN);
  dht20.begin();

  lcd.begin();
  lcd.backlight();
  lcd.clear();

  while (1) {
    int status = dht20.read();
    float t = dht20.getTemperature();
    float h = dht20.getHumidity();

    // Điều kiện lỗi: đọc thất bại hoặc cả 2 giá trị đều bằng 0 hoặc isnan
    if (status != 0 || isnan(t) || isnan(h) || (t == 0.0 && h == 0.0)) {
      glob_sensor_error = true;
      Serial.println("[CANH BAO] Cam bien DHT20 bi loi!");

      lcd.setCursor(0, 0);
      lcd.print("! SENSOR ERROR !");
      lcd.setCursor(0, 1);
      lcd.print("Check DHT20 I2C ");
    } else {
      glob_sensor_error = false;
```

```
      glob_temperature = t;
      glob_humidity = h;

      lcd.setCursor(0, 0);
      lcd.print("Temp: ");
      lcd.print(glob_temperature, 1);
      lcd.print((char)223);
      lcd.print("C   ");

      lcd.setCursor(0, 1);
      lcd.print("Humi: ");
      lcd.print(glob_humidity, 1);
      lcd.print("%   ");
    }
    vTaskDelay(5000);
  }
}

void task_neopixel(void *pvParameters) {
  strip.begin();
  strip.clear();
  strip.show();

  while (1) {
    if (glob_sensor_error) {
      // Khi lỗi: NeoPixel nhấp nháy VÀNG
      strip.setPixelColor(0, strip.Color(255, 200, 0));
      strip.show();
      vTaskDelay(500);

      strip.setPixelColor(0, strip.Color(0, 0, 0));
      strip.show();
      vTaskDelay(500);

    } else {
      // Khi bình thường: trở về quy tắc nhiệt độ (Bài 1)
      float temp = glob_temperature;
      if (temp < 20.0) {
        strip.setPixelColor(0, strip.Color(0, 0, 255));
      } else if (temp < 28.0) {
        strip.setPixelColor(0, strip.Color(0, 255, 0));
      } else {
        strip.setPixelColor(0, strip.Color(255, 0, 0));
      }
      strip.show();
      vTaskDelay(1000);
    }
  }
}

void task_led(void *pvParameters) {
  pinMode(LED_PIN, OUTPUT);

  while (1) {
    if (glob_sensor_error) {
      // Khi lỗi: LED sáng liên tục (Solid ON)
      digitalWrite(LED_PIN, HIGH);
```

```
      vTaskDelay(200);

    } else {
      // Khi bình thường: trở về nhấp nháy theo độ ẩm (Bài 2)
      float humi = glob_humidity;
      uint32_t half_period_ms = (humi < 40.0) ? 1000 : ((humi <= 60.0) ? 500 :
250);

      digitalWrite(LED_PIN, HIGH);
      vTaskDelay(half_period_ms);

      digitalWrite(LED_PIN, LOW);
      vTaskDelay(half_period_ms);
    }
  }
}

void setup() {
  Serial.begin(115200);

  xTaskCreate(task_temp_humi_monitor,
              "Temp_Humi_Monitor",
              4096,
              NULL,
              2,
              NULL
  );

  xTaskCreate(task_neopixel,
              "Task_NeoPixel",
              2048,
              NULL,
              1,
              NULL
  );

  xTaskCreate(task_led,
              "Task_LED_Control",
              2048,
              NULL,
              1,
              NULL
  );
}

void loop() {}
```

BÀI TẬP 4 Bộ lọc trung bình cho nhiệt độ và độ ẩm

Yêu cầu

- Thêm một **bộ lọc trung bình** cho các giá trị cảm biến bằng cách đọc **5 lần**, với khoảng cách **100 ms** giữa 5 mẫu.
- Nếu độ biến thiên của các giá trị nhỏ hơn một **ngưỡng định trước**, cập nhật các biến toàn cục bằng giá trị trung bình.
- Giải thích **ngưỡng** bạn chọn cho bộ lọc (trong khung giải thích).

Lời giải của bạn — mã nguồn hoàn chỉnh:

```
#include <Adafruit_NeoPixel.h>
#include <Arduino.h>
#include <DHT20.h>
#include <LiquidCrystal_I2C.h>
#include <Wire.h>

#define LED_PIN GPIO_NUM_48
#define NEO_PIN 45
#define LED_COUNT 1
#define SDA_PIN 11
#define SCL_PIN 12
#define LCD_ADDR 33

#define NUM_SAMPLES 5
#define SAMPLE_INTERVAL_MS 100
#define THRESHOLD_TEMP_VAR 1.5 // Ngưỡng biến thiên nhiệt độ: 1.5°C
#define THRESHOLD_HUMI_VAR 3.0 // Ngưỡng biến thiên độ ẩm: 3.0%

DHT20 dht20;
LiquidCrystal_I2C lcd(LCD_ADDR, 16, 2);
Adafruit_NeoPixel strip(LED_COUNT, NEO_PIN, NEO_GRB + NEO_KHZ800);

volatile float glob_temperature = 0.0;
volatile float glob_humidity = 0.0;

// Tác vụ cảm biến kết hợp Bộ lọc trung bình và kiểm tra ngưỡng
void task_temp_humi_filter(void *pvParameters) {
  Wire.begin(SDA_PIN, SCL_PIN);
  dht20.begin();

  lcd.begin();
  lcd.backlight();
  lcd.clear();

  while (1) {
    float temp_samples[NUM_SAMPLES];
    float humi_samples[NUM_SAMPLES];
    int valid_count = 0;

    // 1. Đọc 5 mẫu cách nhau 100ms
    for (int i = 0; i < NUM_SAMPLES; i++) {
      if (dht20.read() == 0) {
        temp_samples[valid_count] = dht20.getTemperature();
        humi_samples[valid_count] = dht20.getHumidity();
```

```
        valid_count++;
      }
      vTaskDelay(SAMPLE_INTERVAL_MS);
    }

    if (valid_count == NUM_SAMPLES) {
      float min_t = temp_samples[0], max_t = temp_samples[0], sum_t = 0;
      float min_h = humi_samples[0], max_h = humi_samples[0], sum_h = 0;

      for (int i = 0; i < NUM_SAMPLES; i++) {
        if (temp_samples[i] < min_t) min_t = temp_samples[i];
        if (temp_samples[i] > max_t) max_t = temp_samples[i];
        sum_t += temp_samples[i];

        if (humi_samples[i] < min_h) min_h = humi_samples[i];
        if (humi_samples[i] > max_h) max_h = humi_samples[i];
        sum_h += humi_samples[i];
      }

      float delta_temp = max_t - min_t; // Độ biến thiên nhiệt độ
      float delta_humi = max_h - min_h; // Độ biến thiên độ ẩm

      // 2. So sánh với ngưỡng định trước
      if (delta_temp < THRESHOLD_TEMP_VAR && delta_humi < THRESHOLD_HUMI_VAR)
{
        glob_temperature = sum_t / NUM_SAMPLES; // Cập nhật giá trị trung bình
        glob_humidity = sum_h / NUM_SAMPLES;

        Serial.print("[FILTER OK] T_avg: ");
        Serial.print(glob_temperature, 2);
        Serial.print(" *C | H_avg: ");
        Serial.print(glob_humidity, 2);
        Serial.println(" %");

        lcd.setCursor(0, 0);
        lcd.print("AvgT: ");
        lcd.print(glob_temperature, 1);
        lcd.print((char)223);
        lcd.print("C   ");

        lcd.setCursor(0, 1);
        lcd.print("AvgH: ");
        lcd.print(glob_humidity, 1);
        lcd.print("%   ");
      } else {
        Serial.println("[NOISE REJECTED] Bien thien vuot nguong -> Bo qua mau
nhieu");
      }
    }
    vTaskDelay(4500); // Đảm bảo chu kỳ ~5s
  }
}

void task_neopixel(void *pvParameters) {
  strip.begin();
  strip.clear();
  strip.show();
```

```
  while (1) {
    float temp = glob_temperature;
    if (temp < 20.0) {
      strip.setPixelColor(0, strip.Color(0, 0, 255));
    } else if (temp < 28.0) {
      strip.setPixelColor(0, strip.Color(0, 255, 0));
    } else {
      strip.setPixelColor(0, strip.Color(255, 0, 0));
    }
    strip.show();
    vTaskDelay(1000);
  }
}

void task_led(void *pvParameters) {
  pinMode(LED_PIN, OUTPUT);

  while (1) {
    float humi = glob_humidity;
    uint32_t half_period_ms = (humi < 40.0) ? 1000 : ((humi <= 60.0) ? 500 :
                              250);

    digitalWrite(LED_PIN, HIGH);
    vTaskDelay(half_period_ms);

    digitalWrite(LED_PIN, LOW);
    vTaskDelay(half_period_ms);
  }
}

void setup() {
  Serial.begin(115200);

  xTaskCreate(task_temp_humi_filter,
              "Temp_Humi_Filter",
              4096,
              NULL,
              2,
              NULL
  );
  xTaskCreate(task_neopixel,
              "Task_NeoPixel",
              2048,
              NULL,
              1,
              NULL
  );
  xTaskCreate(task_led,
              "Task_LED_Control",
              2048,
              NULL,
              1,
              NULL
  );
}

void loop() {}
```

Giải thích:

Giải thích ngưỡng bạn chọn và lý do (đơn vị, cách bạn đo độ biến thiên, các đánh đổi):

1. Đơn vị và Ngưỡng lựa chọn:

- Ngưỡng biến thiên nhiệt độ: Delta T = 1.5 °C (độ Celsius).
- Ngưỡng biến thiên độ ẩm: Delta H = 3.0 % (%RH - độ ẩm tương đối).

2. Cách đo độ biến thiên:

- Trong mỗi chu kỳ lọc, hệ thống đọc liên tiếp N = 5 mẫu cảm biến DHT20, mỗi mẫu cách nhau 100 ms (tổng thời gian gom mẫu là 500 ms).

- Độ biến thiên được đo lường bằng độ lệch đỉnh - đỉnh (Peak-to-Peak) trong cụm 5 mẫu:

$$\text{Delta T} = T_{max} - T_{min}$$
$$\text{Delta H} = H_{max} - H_{min}$$

- Tiêu chuẩn chấp nhận: Nếu đồng thời (Delta T < 1.5 °C) và (Delta H < 3.0 %), cụm mẫu được coi là ổn định, vi điều khiển tính trung bình cộng của 5 mẫu và cập nhật vào biến toàn cục glob_temperature, glob_humidity. Nếu vượt ngưỡng, dữ liệu bị xem là nhiễu xung bất thường và bị loại bỏ (giữ nguyên giá trị hợp lệ của chu kỳ trước).

3. Cơ sở khoa học và lý do chọn ngưỡng:

- Quán tính nhiệt vật lý của môi trường: Cảm biến DHT20 và khối không khí môi trường phòng có quán tính nhiệt lớn. Trong khoảng thời gian lấy mẫu cực ngắn 500 ms, nhiệt độ không khí thực tế không thể biến thiên vượt quá 1.5 °C. Bất kỳ bước nhảy nào lớn hơn 1.5 °C đều xuất phát từ nhiễu điện từ trên bus I2C hoặc chập chờn tiếp xúc chân cắm.

- Tương tự, độ ẩm không khí biến thiên chậm do hằng số thời gian bay hơi/ngưng tụ tính bằng hàng chục giây. Thiết lập ngưỡng Delta H = 3.0 % giúp loại bỏ triệt để các bước nhảy giả tạo.

4. Các đánh đổi kỹ thuật (Trade-offs):

- Đánh đổi về Ngưỡng: Nếu chọn ngưỡng quá nhỏ (< 0.5 °C), bộ lọc sẽ từ chối nhầm các thay đổi nhiệt độ thực tế khi có luồng gió mạnh, làm hệ thống bị treo số đo; nếu chọn ngưỡng quá lớn (> 3.0 °C), các xung nhiễu gai sẽ lọt qua làm méo mó giá trị trung bình. Mức 1.5 °C và 3.0 % là điểm cân bằng tối ưu giữa độ nhạy và khả năng kháng nhiễu.

- Đánh đổi về Cửa sổ mẫu N: Chọn N = 5 (thời gian gom 500 ms) giúp hệ thống đáp ứng nhanh theo thời gian thực và tiêu tốn rất ít RAM của ESP32-S3, thay vì chọn N lớn (như N = 20) gây tích lũy độ trễ đo đạc lên tới 2 giây.

BÀI TẬP 5 Đề xuất một bộ lọc nâng cao

Yêu cầu

- Dựa trên các giá trị từ DHT20, hãy **đề xuất một bộ lọc khác** có thể hữu ích cho hệ thống.
- Cung cấp **mã hiện thực** và giải thích các **tham số** bạn chọn cho bộ lọc này.
- Bạn có thể dùng trợ lý AI cho bài tập này.

Lời giải của bạn — mã nguồn hoàn chỉnh:

```
#include <Adafruit_NeoPixel.h>
#include <Arduino.h>
#include <DHT20.h>
#include <LiquidCrystal_I2C.h>
#include <Wire.h>

#define LED_PIN GPIO_NUM_48
#define NEO_PIN 45
#define LED_COUNT 1
#define SDA_PIN 11
#define SCL_PIN 12
#define LCD_ADDR 33

#define FILTER_WINDOW 5
#define SAMPLE_DELAY_MS 100
#define EMA_ALPHA 0.3f // Hệ số làm mượt mũ EMA

DHT20 dht20;
LiquidCrystal_I2C lcd(LCD_ADDR, 16, 2);
Adafruit_NeoPixel strip(LED_COUNT, NEO_PIN, NEO_GRB + NEO_KHZ800);

volatile float glob_temperature = 0.0;
volatile float glob_humidity = 0.0;

// Hàm tính giá trị trung vị (Median Filter) loại bỏ nhiễu xung cực đoan
float compute_median_5(float arr[5]) {
  float sorted[5];
  for (int i = 0; i < 5; i++) sorted[i] = arr[i];

  for (int i = 0; i < 4; i++) {
    for (int j = i + 1; j < 5; j++) {
      if (sorted[i] > sorted[j]) {
        float temp = sorted[i];
        sorted[i] = sorted[j];
        sorted[j] = temp;
      }
    }
  }
  return sorted[2]; // Lấy phần tử chính giữa
}

// Tác vụ Bộ lọc nâng cao: Median Filter kết hợp EMA
void task_advanced_filter(void *pvParameters) {
  Wire.begin(SDA_PIN, SCL_PIN);
  dht20.begin();
```

```
  lcd.begin();
  lcd.backlight();
  lcd.clear();

  float ema_temp = 0.0;
  float ema_humi = 0.0;
  bool is_first_run = true;

  while (1) {
    float raw_temps[FILTER_WINDOW];
    float raw_humis[FILTER_WINDOW];
    int valid_count = 0;

    for (int i = 0; i < FILTER_WINDOW; i++) {
      if (dht20.read() == 0) {
        raw_temps[valid_count] = dht20.getTemperature();
        raw_humis[valid_count] = dht20.getHumidity();
        valid_count++;
      }
      vTaskDelay(SAMPLE_DELAY_MS);
    }

    if (valid_count == FILTER_WINDOW) {
      // Tầng 1: Lọc trung vị để khử 100% nhiễu xung (Spike noise)
      float med_temp = compute_median_5(raw_temps);
      float med_humi = compute_median_5(raw_humis);

      // Tầng 2: Lọc làm mượt hàm mũ (EMA): y[n] = alpha * x[n] + (1 - alpha)
* y[n-1]
      if (is_first_run) {
        ema_temp = med_temp;
        ema_humi = med_humi;
        is_first_run = false;
      } else {
        ema_temp = EMA_ALPHA * med_temp + (1.0f - EMA_ALPHA) * ema_temp;
        ema_humi = EMA_ALPHA * med_humi + (1.0f - EMA_ALPHA) * ema_humi;
      }

      glob_temperature = ema_temp;
      glob_humidity = ema_humi;

      Serial.print("[MEDIAN+EMA] Out_T: ");
      Serial.print(glob_temperature, 2);
      Serial.print(" *C | Out_H: ");
      Serial.print(glob_humidity, 2);
      Serial.println(" %");

      lcd.setCursor(0, 0);
      lcd.print("AdvT: ");
      lcd.print(glob_temperature, 1);
      lcd.print((char)223);
      lcd.print("C   ");

      lcd.setCursor(0, 1);
      lcd.print("AdvH: ");
      lcd.print(glob_humidity, 1);
      lcd.print("%   ");
```

```
    }

    vTaskDelay(4500);
  }
}

void task_neopixel(void *pvParameters) {
  strip.begin();
  strip.clear();
  strip.show();

  while (1) {
    float temp = glob_temperature;
    if (temp < 20.0) {
      strip.setPixelColor(0, strip.Color(0, 0, 255));
    } else if (temp < 28.0) {
      strip.setPixelColor(0, strip.Color(0, 255, 0));
    } else {
      strip.setPixelColor(0, strip.Color(255, 0, 0));
    }
    strip.show();
    vTaskDelay(1000);
  }
}

void task_led(void *pvParameters) {
  pinMode(LED_PIN, OUTPUT);

  while (1) {
    float humi = glob_humidity;
    uint32_t half_period_ms = (humi < 40.0) ? 1000 : ((humi <= 60.0) ? 500 :
250);

    digitalWrite(LED_PIN, HIGH);
    vTaskDelay(half_period_ms);

    digitalWrite(LED_PIN, LOW);
    vTaskDelay(half_period_ms);
  }
}

void setup() {
  Serial.begin(115200);

  xTaskCreate(task_advanced_filter,
              "Task_Adv_Filter",
              4096,
              NULL,
              2,
              NULL
  );

  xTaskCreate(task_neopixel,
              "Task_NeoPixel",
              2048,
              NULL,
              1,
```

```
                NULL
  );

  xTaskCreate(task_led,
                "Task_LED_Control",
                2048,
                NULL,
                1,
                NULL
  );
}

void loop() {}
```

Giải thích:

1. Mô tả bộ lọc đề xuất:
Đề xuất giải pháp Bộ lọc kết hợp 2 tầng: "Bộ lọc Trung vị kết hợp Bộ lọc Trung bình động hàm mũ" (Median Filter + Exponential Moving Average - EMA Filter).
- Tầng 1 (Lọc trung vị phi tuyến - Median Filter): Thu thập cụm 5 mẫu thô (mỗi mẫu cách nhau 100 ms), tiến hành sắp xếp 5 mẫu theo thứ tự tăng dần x(0) <= x(1) <= x(2) <= x(3) <= x(4) và lấy giá trị nằm chính giữa: x_med = x(2).

- Tầng 2 (Lọc làm mượt hàm mũ - EMA Filter): Tín hiệu trung vị x_med tiếp tục được đưa qua phương trình đệ quy hàm mũ bậc nhất:

y[k] = alpha * x_med[k] + (1 - alpha) * y[k-1]

(Trong đó y[k] là giá trị ngõ ra chu kỳ hiện tại, y[k-1] là ngõ ra chu kỳ trước, alpha là hệ số làm mượt).

2. Mục đích của bộ lọc:
- Khắc phục nhược điểm chí mạng của Bộ lọc trung bình cộng ở Bài 4: Bộ lọc trung bình cộng cực kỳ nhạy cảm với ngoại lai (Outliers); một xung nhiễu gai biên độ lớn (ví dụ cáp I2C bị nhiễu nhảy vọt lên 70 °C) sẽ kéo lệch hoàn toàn giá trị trung bình. Việc hủy bỏ mẫu ở Bài 4 gây mất mát toàn bộ 5 mẫu (làm đóng băng hệ thống trong 500 ms). Tầng Median Filter loại bỏ 100% xung nhiễu gai bằng cách đẩy giá trị ngoại lai về biên, giữ lại mẫu thực ở giữa mà không làm mất mẫu.

- Tối ưu hóa bộ nhớ và độ mịn đường đặc tuyến: Tầng EMA tạo đường đặc tuyến phẳng, triệt tiêu hiện tượng răng cưa trên màn hình LCD. Khác với bộ lọc trung bình trượt thông thường phải lưu mảng dữ liệu, EMA chỉ cần lưu duy nhất 1 biến trạng thái quá khứ y[k-1] trong RAM (độ phức tạp bộ nhớ O(1)), tối ưu tuyệt đối cho hệ thống nhúng IoT.

3. Các tham số lựa chọn kèm lý do:
- Kích thước cửa sổ lọc Trung vị: N = 5 mẫu (khoảng cách lấy mẫu Delta t = 100 ms).
+ Lý do: N = 5 là số lẻ nhỏ nhất cho phép triệt tiêu đồng thời tới 2 xung nhiễu ngoại lai xuất hiện liên tiếp (floor(N/2) = 2). Thuật toán sắp xếp 5 phần tử chỉ tốn khoảng 10 phép so sánh, thời gian tính toán dưới 5 micro-giây, hoàn toàn không gây nghẽn CPU.

- Hệ số làm mượt hàm mũ: alpha = 0.3 (với chu kỳ cập nhật 500 ms).
+ Lý do về thời gian đáp ứng: Hằng số thời gian tương đương tau = Delta t / alpha = 0.5s / 0.3 = 1.67 giây. Thời gian trễ này hoàn toàn tương thích với tốc độ biến thiên nhiệt ẩm tự nhiên, không gây cảm giác trễ hiển thị cho người quan sát.

+ Lý do về khả năng triệt tiêu phương sai nhiễu: Tỷ số suy giảm phương sai nhiễu ngẫu nhiên của EMA là alpha / (2 - alpha) = 0.3 / 1.7 = 0.176. Nghĩa là bộ lọc giúp khử tới 82.4% phương sai nhiễu ngẫu nhiên, giúp số liệu nhiệt ẩm hiển thị trên LCD và màu đèn NeoPixel luôn ổn định tuyệt đối, không bị chập chờn rung nhấp nháy.